

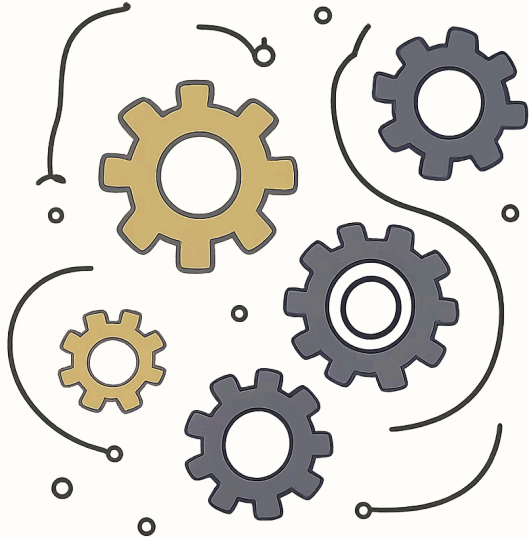


Worker Service в .NET

Фоновые сервисы для обработки задач по расписанию или очереди.
Введение в Worker-сервисы на C#.

ПОСЛЕДНЕЕ ОБНОВЛЕНИЕ: 26.11.2023

Что такое Worker-сервисы?



Помимо консольных и графических приложений на C#, мы можем разрабатывать **сервисы** — приложения, которые работают как фоновый процесс и выполняют определённую работу.

Примеры использования

Фоновая обработка задач по очереди или через промежутки времени

Аналоги

Службы Windows — классический пример подобных сервисов

Создание проекта

Visual Studio и .NET CLI предоставляют специальные шаблоны для создания сервисов.

Visual Studio

Шаблон **Worker Service** доступен при создании нового проекта

.NET CLI

Команда `dotnet new worker` создаёт проект сервиса

Имя проекта

По умолчанию проект называется `WorkerServiceApp`



Структура проекта



→ `Properties/launchSettings.json`

Определяет параметры запуска проекта

→ `appsettings.json`

Конфигурация приложения; `appsettings.Development.json` — для стадии разработки

→ `Program.cs`

Точка входа в программу, неявно определяет класс `Program`

→ `Worker.cs`

Класс сервиса, по умолчанию называется `Worker`

→ `.csproj`

Стандартный файл конфигурации проекта

Файл конфигурации проекта

Файл `.csproj` использует SDK `Microsoft.NET.Sdk.Worker`, предназначенный специально для создания сервисов. В проект по умолчанию добавлен NuGet-пакет `Microsoft.Extensions.Hosting` для развёртывания и запуска сервисов.

 **SDK:** `Microsoft.NET.Sdk.Worker`

 **Пакет:** `Microsoft.Extensions.Hosting v8.0.0`

 **Секреты:** элемент `<UserSecretsId>` содержит GUID для управления секретами проекта

```
<Project Sdk="Microsoft.NET.Sdk.Worker">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>dotnet-WorkerServiceApp-...
  </UserSecretsId>
</PropertyGroup>
<ItemGroup>
  <PackageReference
    Include="Microsoft.Extensions.Hosting"
    Version="8.0.0" />
</ItemGroup>
</Project>
```

Класс сервиса Worker

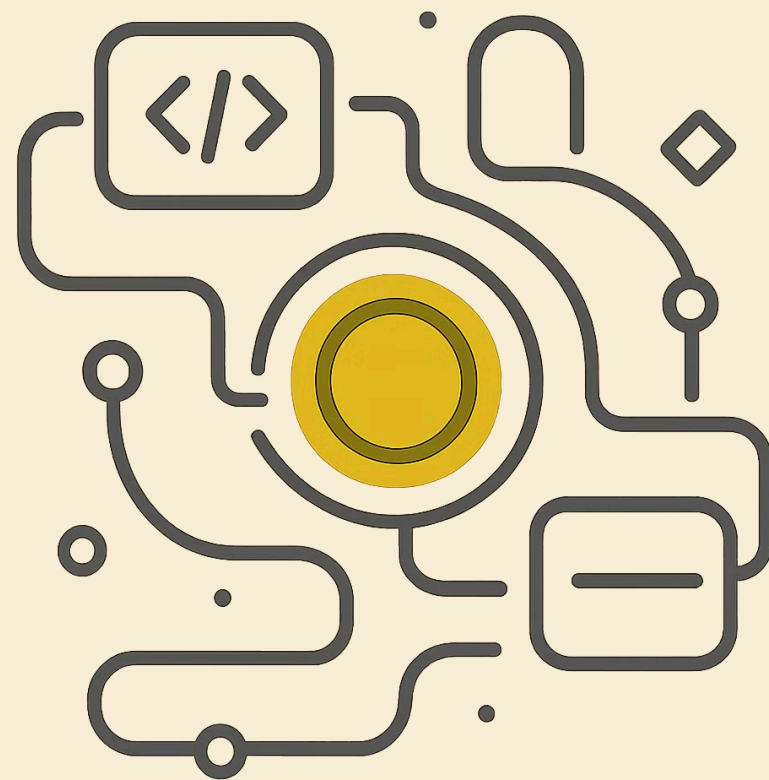
Сервис реализует интерфейс `IHostedService`. Класс `Worker` наследуется от `BackgroundService` — стандартной реализации этого интерфейса.

Зависимость `ILogger`

Через конструктор сервис получает `ILogger<Worker>` для логгирования. `ILogger` передаётся через систему внедрения зависимостей.

Метод `ExecuteAsync()`

Бесконечный цикл работает, пока не установлен токен отмены. Раз в секунду записывает сообщение в лог через `ILogger`. По умолчанию логгирование идёт на консоль.



Код класса Worker

```
public class Worker : BackgroundService
{
    private readonly ILogger<Worker> _logger;

    public Worker(ILogger<Worker> logger)
    {
        _logger = logger;
    }

    protected override async Task
        ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            _logger.LogInformation(
                "Worker running at: {time}",
                DateTimeOffset.Now);
            await Task.Delay(1000, stoppingToken);
        }
    }
}
```

Ключевые моменты

- Наследование от `BackgroundService`
- Конструктор получает `ILogger` через DI
- Цикл `while` проверяет `CancellationToken`
- Задержка `Task.Delay(1000)` — раз в секунду
- Логгирование через `_logger.LogInformation`

Запуск сервиса: создание хоста

Сервис разворачивается и запускается в рамках объекта `IHost`. Точка входа — файл `Program.cs`, который неявно определяет класс `Program`.



Для создания `IHostBuilder` применяется метод `Host.CreateDefaultBuilder(args)`. Далее через `ConfigureServices()` регистрируется сервис, после чего хост строится методом `Build()` и запускается методом `Run()`.

Код Program.cs

```
using WorkerServiceApp;

IHost host = Host.CreateDefaultBuilder(args)
    .ConfigureServices(services =>
    {
        services.AddHostedService<Worker>();
    })
    .Build();

host.Run();
```

1 Host.CreateDefaultBuilder(args)

Создаёт объект `IHostBuilder` с настройками по умолчанию

2 ConfigureServices + AddHostedService

Регистрирует сервис как синглтон; принимает тип, реализующий `IHostedService`

3 Build() и Run()

Строит объект `IHost` и запускает его вместе со всеми сервисами

Результат запуска

После запуска проекта консоль отображает базовую информацию о хосте, а затем — сообщения логгера от сервиса `Worker`.

Что видим в консоли

Базовая информация о хосте, затем сообщения `ILogger` каждую секунду

Стартовая точка

Простой сервис — отправная точка для работы с фоновыми процессами в `.NET`

Следующие шаги

Создание собственного сервиса с бизнес-логикой, настройка расписания и очередей

